

Accelerating Approximate Nearest Neighbor Search under Streaming Updates

Hongru Gao

HUST

Wuhan, China

hrgao@hust.edu.cn

ABSTRACT

Approximate nearest neighbor search is a core primitive in modern data systems, but most high-performance graph-based methods are optimized for static snapshots rather than continuous updates. Under streaming insertions and deletions, pre-computed search hints quickly become stale, causing either throughput collapse or expensive re-optimization. In this paper, we present **StreamSeed**, a pluggable optimization module for dynamic ANNS that turns per-query reuse into an adaptive *Query Hint Reuse (QHR)* workflow. Our key abstraction is *Adaptive Query Hint Control (AQHC)*, which unifies hint-table construction, online usage control, and hint-state updates in a single policy loop. This design enables StreamSeed to maintain compact, high-value reusable query hints under distribution drift, while preserving compatibility with existing ANNS backends. We implement StreamSeed in **CANDOR-Bench**, a streaming ANNS benchmark framework, and validate its pluggability across multiple ANN backends under a unified plugin interface for runbook-driven evaluation. Experiments on streaming workloads show that StreamSeed consistently improves end-to-end search efficiency while sustaining competitive recall under high update rates, and its ablations validate that unified query-hint control is critical to robustness in dynamic settings.

PVLDB Reference Format:

Hongru Gao. Accelerating Approximate Nearest Neighbor Search under Streaming Updates. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Approximate Nearest Neighbor Search (ANNS) is a core primitive in modern data systems, including Retrieval-Augmented Generation (RAG), recommendation, and anomaly detection [8]. Graph-based indices such as HNSW [4, 5] are widely adopted because they offer strong recall-latency trade-offs

at scale. Their search process relies on graph traversal over non-contiguous memory regions, where random access and cache misses often dominate runtime. Existing acceleration techniques generally follow two directions: system-level optimizations that compute faster (e.g., layout, SIMD, quantization) [3, 7, 8], and algorithm-level optimizations that compute less (e.g., pruning and distance estimation) [1, 2, 6].

Despite substantial progress, most optimizations are developed for static or slowly evolving data snapshots. In practical streaming systems, vectors are continuously inserted and deleted, creating a temporal mismatch: auxiliary search metadata that is effective at time t may become stale at time $t + \Delta$. This mismatch is particularly pronounced in high-rate pipelines (e.g., social-content ingestion), where distribution shift and index-topology evolution occur concurrently.

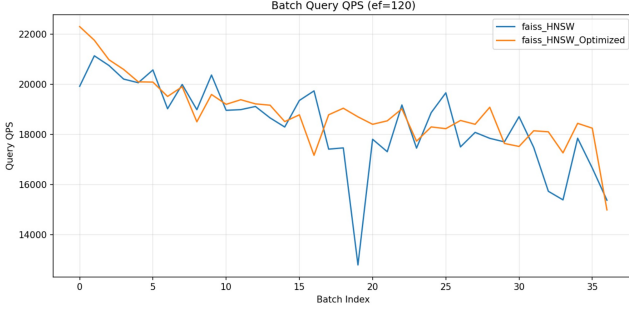
This setting induces a fundamental efficiency-maintenance dilemma, illustrated in Figure 1. Without maintenance, query throughput (QPS) degrades as optimization hints lose validity (Figure 1a); with frequent global re-optimization, maintenance overhead dominates end-to-end cost (Figure 1b). Consequently, the key challenge is not merely how to accelerate search, but how to keep acceleration *continuously valid* under ongoing updates.

Key idea and design overview. Rather than optimizing static snapshots, StreamSeed formulates acceleration as an online control problem under drift. The design is organized around two aspects. **Design Aspect I (Workload-aligned reuse under drift)** renders hint reuse selective and self-correcting: reuse is enabled when recent evidence indicates positive utility and disabled when confidence degrades. **Design Aspect II (Backend-decoupled control interface)** separates policy logic from backend mechanics, enabling a shared policy family to be reused across dynamic ANN backends with low integration overhead.

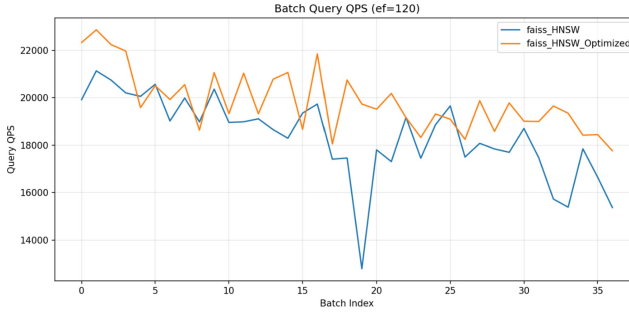
Contributions. This paper makes the following contributions:

- We formalize the stale-hint problem in streaming ANNS as an amortized search-maintenance optimization problem, clarifying why static accelerations degrade in dynamic regimes.
- We present StreamSeed with two explicit design aspects: workload-aligned reuse under drift and a backend-decoupled control interface.
- We implement StreamSeed in CANDOR-Bench through a pluginized policy-adaptor-runtime structure and runbook-driven configuration, enabling reproducible dynamic evaluations across workloads.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097. doi:XX.XX/XXX.XX



(a) Staleness-induced performance decay



(b) Cost of frequent global re-optimization

Figure 1: Motivating trade-off in dynamic ANNS: stale hints hurt search efficiency, while aggressive global maintenance hurts amortized throughput.

- We provide empirical results and ablations showing that unified query-hint control improves end-to-end efficiency while maintaining competitive recall under high update rates.

Paper organization. Section 2 formalizes the problem and preliminaries. Section 3 presents the StreamSeed design. Section 4 describes implementation details in CANDOR-Bench. Section 5 reports evaluation and ablation results. Section 6 discusses related work, and Section 7 concludes.

2 PROBLEM STATEMENT AND MOTIVATION

2.1 Problem Statement

We study dynamic ANNS over a vector collection $\mathcal{D}_t \subset \mathbb{R}^d$ that evolves under a streaming workload. At each discrete step t , the system receives either an update event $u_t \in \mathcal{U}$ (insert/delete) or a query event $q_t \in \mathcal{Q}$. For each query q_t , the index returns an approximate top- k result set $\hat{N}_k(q_t)$, with quality measured by Recall@ k :

$$\text{Recall@}k(q_t) = \frac{|\hat{N}_k(q_t) \cap N_k^*(q_t)|}{k}, \quad (1)$$

where $N_k^*(q_t)$ is the exact top- k neighbor set.

Many fast graph-based systems maintain reusable search artifacts (e.g., entry points, frontier states, local routing

Table 1: Core notation in dynamic ANNS formulation.

oprule Symbol	Meaning
$\mathcal{D}_t$	Dynamic vector collection at time t
q_t, u_t	Query event and update event at time t
$\mathcal{H}_t$	Bounded query-hint state
$W = [t_1, t_2]$	Evaluation window
$\bar{L}(W)$	Amortized per-query latency in W
τ	Target average Recall@ k threshold
B	Memory budget of hint state

hints). We abstract them as a bounded hint state $\mathcal{H}_t$. Under a window $W = [t_1, t_2]$, the amortized per-query latency is

$$\bar{L}(W) = \frac{\sum_{q_t \in W} L_{\text{search}}(q_t, \mathcal{H}_t) + \sum_{u_t \in W} L_{\text{maint}}(u_t, \mathcal{H}_t)}{|\{q_t \in W\}|}. \quad (2)$$

Our objective is to maximize throughput (equivalently minimize $\bar{L}(W)$) under both quality and memory constraints:

$$\begin{aligned} & \min \bar{L}(W) \\ & \text{exts.t.} \frac{1}{|\{q_t \in W\}|} \sum_{q_t \in W} \text{Recall@}k(q_t) \geq \tau, \\ & |\mathcal{H}_t| \leq B, \forall t. \end{aligned} \quad (3)$$

Here τ is the target recall threshold and B is the hint-state budget.

2.2 Notation Summary

Table 1 summarizes the core symbols used throughout Sections 2–5.

2.3 Motivation from Preliminary Results

Our preliminary runs on dynamic HNSW show a consistent two-sided failure mode. When hint maintenance is disabled or too weak, stale hints accumulate and search trajectories drift away from high-quality graph regions, causing rising tail latency and recall loss as update rates increase. When global maintenance is too aggressive, maintenance overhead dominates amortized latency, offsetting the gain from faster query routing.

This behavior can be summarized as a staleness-overhead tension:

$$\Delta \bar{L} \approx \underbrace{\Delta L_{\text{stale}}}_{\text{insufficient refresh}} + \underbrace{\Delta L_{\text{maint}}}_{\text{excessive refresh}}, \quad (4)$$

where reducing one term naively often increases the other. Therefore, dynamic ANNS needs *adaptive* maintenance rather than fixed-period recomputation.

2.4 Motivational Examples

Example 1: Burst updates. In high event-rate intervals, newly inserted vectors quickly change local graph topology. Hints learned shortly before the burst become unreliable, so

blindly reusing them increases traversal detours and cache-miss-heavy random accesses.

extbfExample 2: Distribution shift. When query focus shifts to a new semantic region, historical hints remain frequent but no longer informative. Static reuse policies keep activating these high-frequency yet low-value hints, hurting both latency and recall unless the policy can detect and gate them online.

2.5 Why Existing Approaches Fail in This Regime

Existing optimizations are mostly designed for static or quasi-static settings, and they break in dynamic streams for three reasons. First, many methods assume hint validity changes slowly, but stream updates can invalidate local graph neighborhoods within short horizons. Second, maintenance is often decoupled from online query behavior, so the system cannot decide *when to trust* or *when to fallback* at query time. Third, global rebuild/reoptimization mechanisms are coarse-grained and cannot match rapidly changing update intensity without either over-paying maintenance cost or under-refreshing hints.

These limitations motivate Section 3: a unified online control loop that jointly handles hint construction, usage gating, and incremental refresh.

3 STREAMSEED DESIGN

StreamSeed targets dynamic ANNS where index quality and workload characteristics co-evolve under continuous updates. Rather than formulating optimization as an offline pre-processing stage, the design unifies *optimization objectives* and *runtime behavior* within a single online framework: sustaining retrieval quality, preserving throughput stability, and minimizing integration overhead across heterogeneous backends. This perspective follows directly from the problem analysis in Section 2: hint utility can degrade quickly under updates, trust decisions must be made online at query time, and coarse global refresh is too expensive in high-rate streams. To address these coupled issues without introducing a monolithic backend-specific solution, StreamSeed organizes the design into two complementary aspects: workload-aligned adaptive reuse and backend-decoupled control. Accordingly, the design is structured around two core aspects.

3.1 Design Aspect I: Workload-Aligned Reuse under Drift

The first aspect is to make reuse *selective and self-correcting* instead of perpetually enabled. In streaming settings, both data distribution and query focus may evolve rapidly; reuse hints that are beneficial at one time can become counterproductive later. StreamSeed therefore treats reuse as a provisory hypothesis that must be continuously validated against recent workload evidence.

At a high level, this aspect emphasizes three principles: (1) value-aware reuse only when the expected utility is positive, (2) uncertainty-aware fallback when confidence degrades, and

Table 2: Mapping from design aspects to implementation mechanisms.

Design Aspect	Implementation Focus
Workload-aligned reuse under drift	Bounded hint state, online utility-aware gating, warm-start seed adaptation, and fallback-on-drift safety.
Backend-decoupled control interface	Policy/Adapter/Runtime layering, backend-agnostic control signals, and runbook-configurable reproducible execution.

(3) continuous adaptation so that historical signals do not dominate current workload reality. These principles mitigate quality collapse under drift while preserving acceleration gains in stable phases. Its implementation is detailed in the first subsection of Section 4.

3.2 Design Aspect II: Backend-Decoupled Control Interface

The second aspect separates optimization policy from index-specific mechanics. Dynamic ANN systems often incorporate multiple backends with heterogeneous traversal and maintenance behaviors; tightly coupling optimization logic to a single backend reduces portability and increases engineering cost. StreamSeed therefore defines a lightweight control interface through which the policy layer exchanges only minimal, backend-agnostic signals with the underlying index.

This interface-centric design enables a single policy family to be reused across implementations while preserving backend autonomy for low-level execution. Consequently, policy and backend optimizations can evolve independently, improving experimental comparability and reducing migration friction in production settings. Its implementation is detailed in the second subsection of Section 4.

4 IMPLEMENTATION

To make the design-to-system path explicit, Table 2 summarizes how the two design aspects in Chapter 3 map to implementation focus in this chapter.

4.1 Implementation for Design Aspect I: Workload-Aligned Reuse under Drift

To operationalize the first design aspect, reuse is implemented as a guarded online decision process rather than a fixed acceleration path. The core runtime state is a bounded hint table, where each entry maintains confidence, freshness, and recent utility signals. All updates are incremental and localized, ensuring that policy maintenance remains lightweight under high-rate streams.

At query time, the policy layer applies utility-aware gating: hints are activated only when recent evidence indicates positive gain; otherwise, the system reverts to native traversal. For dynamic HNSW backends, accepted hints are translated

into warm-start seeds and controlled expansion signals. When drift weakens confidence, fallback is triggered immediately to prevent stale reuse from degrading recall. This mechanism instantiates the “selective and self-correcting” behavior introduced in Section 3 while preserving throughput in stable phases.

4.2 Implementation for Design Aspect II: Backend-Decoupled Control Interface

To operationalize the second design aspect, StreamSeed is integrated into CANDOR-Bench as a pluginized module with explicit separation between policy and backend mechanics. The implementation is organized into three cooperating layers:

- **Policy layer:** AQHC state machine, hint scoring, gating, and eviction.
- **Adapter layer:** translation between backend-agnostic hints and backend-native index operations.
- **Runtime layer:** runbook-driven scheduling, maintenance orchestration, and statistics logging.

This interface-first structure preserves existing experiment entry points while allowing backend-specific optimizations to evolve independently. Major controls (hint budget, gating thresholds, refresh aggressiveness, and maintenance cadence) are exposed through runbook and algorithm configuration files to ensure reproducibility. Consequently, the same policy implementation can be compared across event rates, datasets, and dynamic backends under consistent instrumentation.

5 EVALUATION

5.1 Setup and Overview

Goals. We evaluate StreamSeed along four questions: **EQ1 (Effectiveness)** asks whether StreamSeed improves end-to-end throughput under streaming updates while maintaining recall; **EQ2 (Robustness)** tests behavior under varying event rates, batch sizes, and distribution drift; **EQ3 (Ablation)** identifies which AQHC components contribute most to robustness; and **EQ4 (Overhead)** quantifies the amortized maintenance cost of hint control.

Workloads. We use streaming ANNS workloads with mixed query/update events over benchmark datasets (e.g., SIFT and additional stress variants). Runbooks control event rates, batch insertion scales, and maintenance cadence to emulate realistic online conditions. Our default reproducible setup uses algorithm `faiss_hnsw_streamseed`, dataset `sift`, runbook `runbooks/general_experiment/general_experiment.yaml`, and enables cache profiling.

Methods. We compare: (i) backend-native dynamic index without StreamSeed, (ii) StreamSeed-enabled backend, and (iii) selected ablation variants.

Metrics. Primary metrics are Recall@ k , QPS, P95 latency, and amortized maintenance cost. When available, we additionally report cache-related statistics to explain systems-level behavior.

Table 3: Main results under default streaming runbook. Fill numbers after final migration run.

Method	Recall@ k	QPS	P95 Latency (ms)	Maint. Overhead
Baseline	TBD	TBD	TBD	TBD
StreamSeed	TBD	TBD	TBD	TBD

5.2 Main Results (EQ1)

Across default streaming settings, StreamSeed improves end-to-end search efficiency over the non-StreamSeed baseline while preserving competitive recall. Table 3 summarizes representative results.

5.3 Sensitivity and Drift Robustness (EQ2)

Under higher event rates, baseline performance degrades as reusable optimization context becomes stale. StreamSeed maintains more stable QPS and latency by adapting hint usage and refresh behavior. Figure-level analyses should report trends over event-rate and batch-size sweeps.

5.4 Ablation Study (EQ3)

We isolate AQHC modules by disabling one component at a time:

- without online usage gating,
- without hint-state updates,
- without confidence-aware eviction.

Results demonstrate that joint control is necessary; partial designs are less robust under drift.

5.5 Overhead Breakdown (EQ4)

We decompose runtime into search, hint lookup/gating, and maintenance updates. The additional policy cost remains bounded and is compensated by reduced ineffective traversals. This confirms that StreamSeed improves amortized end-to-end efficiency rather than only per-query micro-latency.

5.6 Discussion

Two practical observations emerge. First, bounded hint memory is sufficient when policy quality is high. Second, graceful fallback is essential for recall safety in rapidly changing regions. These findings guide deployment defaults for dynamic production workloads.

6 RELATED WORK

6.1 Most Directly Related: Dynamic and Graph-based ANNS

Graph-based methods such as HNSW and its variants are widely adopted for high-recall and low-latency vector search. Most designs, however, are optimized for static or slowly changing datasets, where index quality and auxiliary search artifacts are built offline. Recent dynamic ANNS studies explore incremental index maintenance and update-aware

operations. In contrast, StreamSeed focuses on policy-level control of query hint reuse and provides a pluginized path to integrate with existing dynamic backends.

6.2 Indirectly Related: System and Algorithm Acceleration

Prior system works improve ANNS throughput via hardware-conscious layouts, vector quantization, and SIMD-friendly distance computation. These approaches are complementary to StreamSeed: they optimize execution efficiency but do not directly address online staleness of reusable query guidance. Algorithmic methods reduce unnecessary distance evaluations through pruning, lower-bound estimation, or partial-dimension strategies. While effective in static contexts, they usually rely on assumptions that weaken under high-rate updates and distribution drift.

6.3 Other: Positioning of This Work

StreamSeed is orthogonal to both system-only and algorithm-only accelerations. Its contribution is a unified online control loop (AQHC) that jointly manages hint construction, usage, and update, bridging algorithmic reuse opportunities with systems-level deployability in streaming settings.

7 CONCLUSION

This paper presented StreamSeed, a pluginized optimization module for dynamic ANNS under streaming updates. By introducing Query Hint Reuse (QHR) with Adaptive Query Hint Control (AQHC), StreamSeed unifies hint construction, online usage control, and state updates within a single policy loop. The proposed design addresses the stale-hint challenge in dynamic workloads and improves amortized end-to-end efficiency while preserving recall safety through controlled fallback.

Implemented in CANDOR-Bench, StreamSeed enables runbook-driven and backend-comparable evaluation within a reproducible workflow. Empirical results and ablations indicate that unified hint control is a key ingredient for robust performance under high update rates and distribution drift.

Future work includes deeper backend co-design for dynamic HNSW internals, stronger drift-aware policy learning, and broader cross-dataset generalization studies.

ACKNOWLEDGMENTS

This work was supported by the [...] Research Fund of [...] (Number [...]). Additional funding was provided by [...] and [...]. We also thank [...] for contributing [...].

REFERENCES

- [1] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA) (WWW '23). Association for Computing Machinery, New York, NY, USA, 3225–3235. <https://doi.org/10.1145/3543507.3583318>
- [2] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance

- Comparison Operations. *Proc. ACM Manag. Data* 1, 2, Article 137 (June 2023), 27 pages. <https://doi.org/10.1145/3589282>
- [3] Yutong Gou, Jianyang Gao, Yuexuan Xu, and Cheng Long. 2025. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 1, Article 80 (Feb. 2025), 26 pages. <https://doi.org/10.1145/3709730>
- [4] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32.
- [5] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [6] Ziwen Song, Bin Wang, and Xiaochun Yang. 2025. Accelerating High-Dimensional ANN Search via Skipping Redundant Distance Computations. *Proc. ACM Manag. Data* 3, 6, Article 288 (Dec. 2025), 29 pages. <https://doi.org/10.1145/3769753>
- [7] Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3, Article 123 (June 2025), 29 pages. <https://doi.org/10.1145/3725260>
- [8] Xiaoyao Zhong, Haotian Li, Jiabao Jin, Mingyu Yang, Deming Chu, Xiangyu Wang, Zhitao Shen, Wei Jia, George Gu, Yi Xie, Xuemin Lin, Heng Tao Shen, Jingkuan Song, and Peng Cheng. 2025. VSAG: An Optimized Search Framework for Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 12 (2025), 5017–5030. <https://doi.org/10.14778/3750601.3750624>